VOGENFLOW

Voice-based Generative AI for Workflows
Complete Engineering Specification, Architecture, Decision Rationale, Signal Processing Mechanics, RAG Grounding & QA Blueprints

---

Author / Architect: Antigravity Systems & Core Engineering
Document Version: 1.0.0 (Production Master)
Target Platform: Windows / macOS / Linux (PyQt6 & Python 3.10+)
STT & LLM Engines: Groq Whisper LPU & Llama-3.3-70B / Local Ollama
Context Retrieval: Embedded Okapi BM25 RAG Engine
Date: August 2026

Table of Contents

Chapter 1: Product Requirements Document (PRD)

Product Requirements Document (PRD): Voice-to-Insight & Blueprint Generator

1. Executive Summary & Problem Statement

Engineers and technical founders spend hours translating high-level spoken brainstorming, meeting notes, and voice memos into structured technical specifications (PRDs, Architecture documents, Task lists, and Sequence flows).

Voice-to-Insight is an AI-powered desktop and CLI automation engine that ingests live or recorded speech, transcribes it in sub-second latency using Groq Whisper LPUs, grounds the context against local project codebases using an embedded RAG engine, and automatically generates production-ready engineering documentation suites.

2. Goals & Success Metrics (KPIs)

- **Transcription Latency**: $< 500\text{ ms}$ for 60 seconds of audio via Groq Whisper LPU.
- **Specification Accuracy**: Zero hallucinations on existing APIs/schemas when grounded with the RAG engine.
- **Time to Blueprint**: $< 10\text{ seconds}$ to generate a full 6-document engineering specification suite from a raw voice recording.
- **User Interface Responsiveness**: Continuous 60 FPS UI during audio capture and background network operations.

3. User Personas & Use Cases

1. The Solo Architect / Indie Hacker: Wants to speak an idea aloud while driving or walking, then click one button to generate a complete `PRD.md`, `architecture.md`, `flow.md`, and `tasks.md` ready to implement.

2. The Engineering Lead: Records a team design debate or sprint planning session, indexes the current codebase, and generates grounded technical task lists with acceptance criteria.

3. The Developer (CLI Power User): Uses terminal commands (`python main.py generate --text "..."`) in automation scripts or Git commit hooks.

4. Functional Requirements

P0 (Critical / Core MVP)

- **Live Microphone Recording**: Non-blocking audio capture with real-time waveform visualizer, pause, resume, and stop controls.
- **File Upload & Drag-and-Drop**: Support for `.wav`, `.mp3`, `.m4a`, `.ogg`, and `.flac`.
- **Groq Whisper STT Integration**: Ultra-fast cloud speech-to-text with timestamp and segment extraction.
- **Full Blueprint Suite Synthesis**: Generation of `PRD.md`, `architecture.md`, `flow.md`, `tech_stack.md`, `tasks.md`, and `implementation_plan.md`.
- **Direct Repository Export**: Saving generated documents directly into user-selected target directories.

P1 (Important)

- **Embedded RAG Engine**: Indexing local codebases (`.py`, `.md`, `.ts`, `.json`), query decomposition, and BM25 grounded context retrieval.
- **Ollama Local LLM Fallback**: Ability to run completely offline without internet or cloud credits.
- **Multi-Tab Markdown Editor**: Review, edit, and copy generated documents in individual tabs.
- **PDF Compilation**: Generating a combined, styled master `.pdf` file of all documentation.

P2 (Nice-to-Have / Future)

- **Speaker Diarization**: Multi-speaker identification for meeting notes.
- **Voice-Back Audio Feedback (TTS)**: Conversational audio responses via `edge-tts` or ElevenLabs.

---

5. Non-Functional Requirements

- **Performance**: Instant UI feedback; memory footprint < *150\text{ MB}*.
- **Security**: No hardcoded API keys; dynamic masked key input; local-only processing for audio recordings.
- **Reliability**: Graceful degradation when network drops; informative user alerts without application crashes.

Chapter 2: Engineering Decisions & Architectural Rationale

Engineering Decisions & Architectural Rationale

This document provides a line-by-line, file-by-file account of all architectural choices, engineering trade-offs, and implementation strategies used across the **Voice-to-Insight** codebase.

---

1. File-by-File Technical Decisions

`src/config.py`

- **Decision**: Centralized environment variable management with fallback paths and auto-creation of runtime folders (`output/`, `docs/`, `temp_audio/`).
- **Why**: Prevents scattered `os.getenv()` calls across multiple modules. When modules import `config.py`, base paths and directories are guaranteed to exist, preventing `FileNotFoundError` or permission issues during runtime recording.
- **Key Implementation Details**: Uses `python-dotenv` with graceful fallbacks. Includes a helper `has_groq_key()` to detect placeholder vs. active keys.

`src/audio_recorder.py`

- **Decision**: Built a non-blocking, thread-safe audio recorder using `sounddevice.InputStream` with 16-bit PCM integer sampling at 16,000 Hz mono.
- **Why**:
- **16 kHz Mono**: Human speech frequencies are between 80 Hz and 8 kHz (Nyquist theorem requires 16 kHz). Higher sample rates (44.1 kHz or 48 kHz) waste memory and network bandwidth without improving Whisper's word error rate.
- **Thread-Safety (`threading.Lock`)**: Prevents race conditions between audio streaming callbacks (which run on low-level OS threads) and UI control events (which run on the PyQt event loop).
- **Real-Time Amplitude Callbacks**: The internal `_audio_callback` calculates Root Mean Square (RMS) amplitude:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

This provides low-latency volume metrics directly to the UI visualizer without blocking the audio driver.

`src/stt_service.py`

• **Decision**: Implemented an Abstract Factory / Provider pattern with `BaseSTTProvider`, `GroqWhisperProvider`, and a unified `STTService` facade.

• **Why**:

• Decouples the UI and insight layers from specific speech APIs.

• Groq's Tensor Streaming Processors (LPUs) transcribe audio in **sub-second latency (200–400ms)** vs 10–30 seconds on local CPU.

• Returns a standardized `STTResult` object containing text, duration, inference latency, detected language, and timestamped segments.

`src/rag_engine.py`

• **Decision**: Implemented a self-contained, lightweight BM25 retrieval engine with **Spoken Query Decomposition** and **Markdown Header-Aware Chunking**.

• **Why**:

• **Why RAG for Voice?**: Spontaneous spoken brainstorming lacks formal structure. Users say: *"Connect the frontend to the payment service we created earlier."* RAG indexes local repositories (`.py`, `.md`, `.ts`, `.json`), retrieves exact function signatures, schemas, and architecture rules, and grounds the LLM prompt in factual code context.

• **Query Decomposition**: A 2-minute voice recording frequently covers multiple disparate topics (e.g. database schema, auth tokens, deployment). The engine extracts key domain terms and splits multi-sentence transcripts into focused sub-queries, searches the index, and aggregates the top scoring chunks.

• **Okapi BM25 Scoring**:

$$\text{Score}(D, Q) = \sum_{q \in Q} \text{IDF}(q) \cdot \frac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

BM25 provides exact keyword and symbol matching for code identifiers (function names, API paths) where semantic embeddings often lose exact string fidelity.

`src/insight_engine.py`

• **Decision**: Engineered specialized, multi-stage system prompts for 6 distinct engineering artifacts: `PRD.md`, `architecture.md`, `flow.md`, `tech_stack.md`, `tasks.md`, and `implementation_plan.md`.

• **Why**:

• Rather than generating a single monolithic text blob, modular generation allows targeted refinement.

• Incorporates strict markdown guidelines, Mermaid syntax constraints, and Agile task checklists (`- [ ]`).

• Supports switching between **Groq (`llama-3.3-70b-versatile`)** for ultra-fast generation and **Ollama** for offline privacy.

`src/export_service.py`

- **Decision**: Decoupled document persistence into a standalone static utility service.
- **Why**: Enables one-click exports directly into the root or `/docs` folder of target user repositories, facilitating seamless Git commits.

`src/ui/audio_visualizer.py`

- **Decision**: Created a custom `QWidget` utilizing `QPainter` with antialiased rounded rectangles, linear gradients, and an internal decay timer.
- **Why**: Provides fluid, dynamic visual feedback when the user is speaking. The visualizer applies a Gaussian-like bell curve from center bars with subtle jitter to create a natural frequency spectrum aesthetic.

`src/ui/main_window.py`

- **Decision**: Built a 3-pane responsive desktop UI using PyQt6 with background worker threads (`QThread`) and `WorkerSignals`.
- **Why**:
- **Worker Threads**: Network I/O (Groq API) and file processing are executed off the main Qt UI thread, ensuring the GUI remains 60fps responsive without stuttering or "Not Responding" freezes.
- **Drag-and-Drop (`DropZoneWidget`)**: Native OS drag-and-drop handles `.wav`, `.mp3`, `.m4a`, `.ogg`, and `.flac` files directly from Explorer.
- **Tabbed Editor**: Each generated artifact can be reviewed, edited, or copied in isolated tabs with monospace formatting.

`src/cli.py` & `main.py`

- **Decision**: Dual-mode entry point that auto-detects CLI arguments vs GUI execution.
- **Why**: Gives developers the flexibility to run automated batch transcriptions and blueprint generations in CI/CD scripts or terminal environments while providing the full GUI for interactive brainstorming.

Chapter 3: System Execution Flow & Code Lifecycle

System Execution Flow & Code Lifecycle (Flow.md)

This document maps every feature, entry point, execution order, function call hierarchy, and data transformation pipeline across the **Voice-to-Insight** system.

---

1. System Entry Points

```
flowchart TD
    Start(["User Launches Application"]) --> CheckArgs{"CLI Arguments Provided?"}

    CheckArgs -- Yes --> RunCLI["src/cli.py: main()"]
    CheckArgs -- No --> RunGUI["src/ui/main_window.py: run_gui()"]

    subgraph CLI_Path ["CLI Execution Routes"]
        RunCLI --> CMD_Record["cmd: record -> AudioRecorder"]
        RunCLI --> CMD_Transcribe["cmd: transcribe -> STTService"]
        RunCLI --> CMD_Generate["cmd: generate -> RAG + InsightEngine"]
    end

    subgraph GUI_Path ["GUI Desktop Route"]
        RunGUI --> QAppInit["Initialize QApplication & Dark Theme"]
        QAppInit --> WinInit["MainWindow.__init__()"]
        WinInit --> RenderUI["Render Audio, Transcript & Blueprint Panels"]
    end
```

---

2. End-to-End Execution Sequence

Workflow A: Live Speech Recording & Transcription Flow

```
sequenceDiagram
    autonumber
    actor User
    participant UI as MainWindow (Qt Event Loop)
    participant Rec as AudioRecorder (Worker Thread)
    participant SD as sounddevice (Audio Driver)
    participant Vis as AudioVisualizer Widget
    participant STTWorker as TranscriptionWorker (QThread)
    participant STT as GroqWhisperProvider
    participant GroqAPI as Groq Cloud LPU

    User->>UI: Clicks "■■ Start Recording"
```

```
    UI->>Rec: start()
    Rec->>SD: Open InputStream(16kHz, int16)

    loop Every Audio Frame (1024 samples)
        SD-->>Rec: _audio_callback(indata)
        Rec->>Rec: Append frame to buffer & compute RMS
        Rec-->>Vis: set_level(normalized_rms)
        Vis->>Vis: Trigger QPainter repaint (waveform animation)
    end

    User->>UI: Clicks "■ Stop Recording"
    UI->>Rec: stop()
    Rec->>SD: Stop & Close InputStream
    UI->>Rec: save_wav("temp_audio/recording_xyz.wav")
    Rec-->>UI: Return saved WAV filepath

    User->>UI: Clicks "■ Transcribe Audio to Text"
    UI->>STTWorker: start() (Spawns QThread)
    STTWorker->>STT: transcribe_file(audio_path, model)
    STT->>GroqAPI: POST /v1/audio/transcriptions (multipart/form-data)
    GroqAPI-->>STT: Return JSON { text, segments, duration }
    STT-->>STTWorker: Return STTResult object
    STTWorker-->>UI: signals.finished.emit(STTResult)
    UI->>UI: Populate transcript_edit with text
    UI->>UI: Update word count, duration, and latency metrics
```

Workflow B: RAG Context Retrieval & Insight Generation Flow

```
sequenceDiagram
    autonumber
    actor User
    participant UI as MainWindow
    participant RAG as LocalRAGEngine
    participant GenWorker as InsightGenerationWorker (QThread)
    participant Engine as InsightEngine
    participant GroqLLM as Groq Cloud API (Llama-3.3-70B)
    participant Exporter as ExportService

    opt Index Codebase for RAG Grounding
        User->>UI: Clicks "■ Index Target Codebase..."
        UI->>RAG: index_directory(selected_repo_path)
        RAG->>RAG: Scan .py, .md, .ts, .json files
        RAG->>RAG: Section & fixed-window chunking
        RAG->>RAG: Build token inverted index & compute BM25 IDF stats
        RAG-->>UI: Return indexed file & chunk counts
    end

    User->>UI: Clicks "■ Generate All Blueprints"
    UI->>RAG: retrieve_grounded_context(transcript)
    RAG->>RAG: decompose_query(transcript) -> [subquery1, subquery2, ...]
    RAG->>RAG: search(subqueries) -> Rank chunks using Okapi BM25
    RAG-->>UI: Return formatted grounded context markdown block

    UI->>GenWorker: start(transcript, rag_context, doc_type="all")

    par Parallel/Sequential LLM Syntheses
        GenWorker->>Engine: generate_prd(transcript, rag_context)
        Engine->>GroqLLM: POST /v1/chat/completions (PRD Prompt)
```

```
    GroqLLM-->>Engine: Return PRD.md content
and
    GenWorker->>Engine: generate_architecture(transcript, rag_context)
    Engine->>GroqLLM: POST /v1/chat/completions (Architecture Prompt)
    GroqLLM-->>Engine: Return architecture.md content
and
    GenWorker->>Engine: generate_flow(transcript, rag_context)
    Engine->>GroqLLM: POST /v1/chat/completions (Flow Prompt)
    GroqLLM-->>Engine: Return flow.md content
and
    GenWorker->>Engine: generate_tech_stack(transcript, rag_context)
    Engine->>GroqLLM: POST /v1/chat/completions (Tech Stack Prompt)
    GroqLLM-->>Engine: Return tech_stack.md content
and
    GenWorker->>Engine: generate_tasks(transcript, rag_context)
    Engine->>GroqLLM: POST /v1/chat/completions (Tasks Checklist Prompt)
    GroqLLM-->>Engine: Return tasks.md content
and
    GenWorker->>Engine: generate_implementation_plan(transcript, rag_context)
    Engine->>GroqLLM: POST /v1/chat/completions (Implementation Plan Prompt)
    GroqLLM-->>Engine: Return implementation_plan.md content
end

GenWorker-->>UI: signals.finished.emit(all_generated_docs)
UI->>UI: Populate each QTabWidget document tab

opt Export to User Repository
    User->>UI: Clicks "■ Export All to Repository..."
    UI->>Exporter: export_documents(docs, target_dir)
    Exporter->>Exporter: Write files to target folder
    Exporter-->>UI: Return list of saved file paths
end
```

3. Function Call Hierarchy Matrix

User Trigger	Initiating Function	Called Methods & Hierarchy	Destination Artifact / State
Click "Start Recording"	`_toggle_recording()`	`AudioRecorder.start()` *\to* `sd.InputStream.start()`	Streaming microphone PCM buffer
Audio Chunk Arrives	`sd` driver	`_audio_callback()` *\to* `AudioVisualizer.set_level()` *\to* `QPainter.paintEvent()`	Real-time animated waveform UI
Click "Stop Recording"	`_toggle_recording()`	`AudioRecorder.stop()` *\to* `AudioRecorder.save_wav()`	`temp_audio/recording_*.wav`
Drop Audio File	`DropZoneWidget.dropEvent()`	`MainWindow._on_file_selected()`	`current_audio_file` state updated

Click "Transcribe"	`_run_transcription()`	`TranscriptionWorker.start()` *to* `STTService.transcribe_file()` *to* `Groq.audio.transcriptions.create()`	Populates `transcript_edit` with cleaned text
Click "Index Codebase"	`_index_codebase_dir()`	`LocalRAGEngine.index_directory()` *to* `index_file()` *to* `_recompute_bm25_stats()`	In-memory BM25 index of project
Click "Generate All"	`_run_generation("all")`	`LocalRAGEngine.retrieve_grounded_context()` *to* `InsightGenerationWorker.start()` *to* `InsightEngine.generate_all()`	Populates all 6 tabs (`PRD`, `Arch`, `Flow`, `Tech`, `Tasks`, `Plan`)
Click "Export to Repo"	`_export_to_directory()`	`ExportService.export_documents()`	Writes `.md` files to user's Git directory
Click "Export PDF"	`_generate_pdf_report()`	`generate_master_pdf()` (ReportLab)	`Voice_Insights_Master_Documentation.pdf`

Chapter 4: System Architecture & Component Specifications

System Architecture Specification (Architecture.md)

This document describes the high-level and component architecture of the **Voice-to-Insight System**, its layer boundaries, data models, concurrency paradigms, and security boundaries.

---

1. High-Level Architecture (C4 Container View)

```
flowchart TD
    subgraph Client_Desktop ["PyQt6 Desktop Presentation Layer"]
        UI_Audio["Audio Panel (Recorder / Visualizer / File Drop)"]
        UI_Transcript["Transcript & RAG Context Panel"]
        UI_Tabs["Blueprint & Markdown Document Tabs"]
    end

    subgraph Service_Layer ["Python Core Business Logic & Orchestration"]
        RecorderService["AudioRecorder (sounddevice Engine)"]
        STTServiceFacade["STTService (Groq Whisper Client)"]
        RAGService["LocalRAGEngine (BM25 Inverted Index)"]
        InsightService["InsightEngine (Prompt Synthesizer)"]
        ExportEngine["ExportService (File Serializer)"]
    end

    subgraph AI_Cloud_Infrastructure ["Inference Engines"]
        GroqLPU_STT["Groq Whisper LPU (whisper-large-v3-turbo)"]
        GroqLPU_LLM["Groq Llama-3.3-70B LPU"]
        OllamaLocal["Local Ollama Service (Fallback)"]
    end

    subgraph Storage_Layer ["Local Filesystem Storage"]
        TempAudio["temp_audio/*.wav"]
        OutputDir["output/*.md"]
        TargetUserRepo["User Target Repository"]
        MasterPDF["output/Voice_Insights_Master_Documentation.pdf"]
    end

    UI_Audio --> RecorderService
    RecorderService --> TempAudio
    UI_Audio --> STTServiceFacade
    STTServiceFacade --> GroqLPU_STT
    STTServiceFacade --> UI_Transcript

    UI_Transcript --> RAGService
    RAGService --> InsightService
    UI_Transcript --> InsightService

    InsightService --> GroqLPU_LLM
    InsightService --> OllamaLocal
    InsightService --> UI_Tabs
```

```
    UI_Tabs --> ExportEngine
    ExportEngine --> OutputDir
    ExportEngine --> TargetUserRepo
    ExportEngine --> MasterPDF
```

---

2. Component Breakdown & Responsibilities

Component	Primary Class	Key Responsibility	Threading Model
Audio Recorder	`AudioRecorder`	Captures microphone input, buffers frames, calculates RMS amplitude.	Native OS Audio Callback + Mutex Lock
Visualizer	`AudioVisualizer`	Renders dynamic multi-bar animated waveform UI with frequency jitter.	Qt Main UI Thread (30ms QTimer)
STT Engine	`STTService` / `GroqWhisperProvider`	Transcribes audio files into structured `STTResult` via Groq Whisper.	`TranscriptionWorker` (`QThread`)
RAG Knowledge Base	`LocalRAGEngine`	Scans local codebase, parses chunks, indexes inverted tokens, computes Okapi BM25 scores, decomposes voice queries.	Synchronous / Background task
Insight Synthesizer	`InsightEngine`	Orchestrates specialized system prompts to generate PRDs, Architecture docs, Flow diagrams, and Task lists.	`InsightGenerationWorker` (`QThread`)
Export Service	`ExportService`	Serializes generated markdown files directly into target project folders.	Synchronous File I/O

---

3. Concurrency & Threading Architecture

To guarantee a responsive 60 FPS user interface without freeze-ups during network calls or large file reads, the system uses a clean multi-threaded architecture:

```
stateDiagram-v2
    [*] --> IdleState : Application Launch

    state "Main UI Thread (Qt Event Loop)" as UIThread {
        IdleState --> RecordingState : User clicks Record
        RecordingState --> ProcessingAudio : User clicks Stop
        ProcessingAudio --> TranscribingState : User clicks Transcribe
        TranscribingState --> EditingTranscript : Worker finished signal
```

```
        EditingTranscript --> GeneratingInsights : User clicks Generate
        GeneratingInsights --> DisplayingDocs : Worker finished signal
    }

    state "Background QThreads" as WorkerThreads {
        state "AudioRecorder Thread" as RecThread
        state "TranscriptionWorker (QThread)" as STTThread
        state "InsightGenerationWorker (QThread)" as LLMThread
    }

    RecordingState --> RecThread : Stream PCM frames
    TranscribingState --> STTThread : Groq API HTTP POST
    GeneratingInsights --> LLMThread : Groq / Ollama API HTTP POST

    STTThread --> UIThread : emit(finished / error)
    LLMThread --> UIThread : emit(finished / error)
```

---

4. Data Models & Interface Contracts

`STTResult` Data Model

```
class STTResult:
    text: str                    # Clean transcribed text
    duration_seconds: float      # Total duration of source audio
    latency_seconds: float       # Time taken by Groq LPU to transcribe
    language: str                # Detected ISO language code (e.g. "en")
    segments: list               # Timestamped word/phrase segments
    provider: str                # "groq" or "local"
    model: str                   # "whisper-large-v3-turbo"
```

`DocumentChunk` Data Model (RAG)

```
class DocumentChunk:
    source_path: str             # Absolute or relative path to source file
    chunk_id: int                # Sequence index within the file
    content: str                 # Raw text or code chunk
    title: str                   # File basename or section header
    tokens: list[str]            # Normalized token array for BM25 matching
```

---

5. Security & Privacy Architecture

1. **Zero Hardcoded Secrets**: All API keys are loaded via `.env` or input dynamically through a masked Qt dialog (`QLineEdit.EchoMode.Password`) in memory.

2. **Local Processing of Audio Buffers**: Live recording buffers are stored in a private local directory (`temp_audio/`) and never uploaded to any intermediary third-party servers other than the designated Groq endpoint.

3. Offline Privacy Option: Users working with confidential codebases can switch the provider to **Ollama**, ensuring zero telemetry or data leaves their local workstation.

Chapter 5: Technology Stack & Environment Setup

Technology Stack & Environment Specification (tech_stack.md)

This document specifies the technology stack choices, dependency versions, hardware requirements, and setup instructions for the **Voice-to-Insight System**.

---

1. Core Technology Stack

Layer	Technology	Version	Rationale
Language	Python	`3.10+` / `3.12`	Broad ecosystem support for audio processing, ML SDKs, Qt bindings, and file manipulation.
GUI Framework	PyQt6	`^6.6.1`	Native OS desktop performance, robust multi-threading via `QThread`, custom `QPainter` visualizers, and responsive layout management.
UI Theme	PyQtDarkTheme	`^0.1.7`	Modern, cohesive dark mode design system across all standard Qt widgets.
Audio Capture	sounddevice	`^0.5.6`	Cross-platform PortAudio bindings for low-latency live microphone streaming and buffer collection.
Audio Processing	numpy / scipy / wave	`^1.26.0`	High-speed PCM array manipulation, RMS power computation, and 16-bit WAV encoding.
Cloud STT	Groq Whisper API	`whisper-large-v3-turbo`	Sub-second audio transcription (200–400ms) with zero local GPU VRAM requirements.
Local / Cloud LLM	Groq Cloud / Ollama	`llama-3.3-70b` / `llama3.2`	High-throughput specification synthesis, complex Mermaid diagram generation, and offline fallback.
RAG Retrieval	BM25 Inverted Index	Custom / zero-dep	Fast, zero-configuration local semantic retrieval, query decomposition, and code grounding.

PDF Generation	ReportLab	`^4.3.1`	High-fidelity PDF compilation with custom styles, headers, tables of contents, and cover pages.

2. Dependency Matrix (`requirements.txt`)

```
groq>=0.18.0
PyQt6>=6.6.1
PyQtDarkTheme>=0.1.7
sounddevice>=0.4.6
numpy>=1.26.0
scipy>=1.12.0
python-dotenv>=1.0.1
requests>=2.31.0
reportlab>=4.1.0
markdown>=3.6
pydantic>=2.6.0
```

3. Environment Variables Configuration

Variable Name	Required?	Default Value	Description
`GROQ_API_KEY`	Recommended	`""`	API key from [Groq Console](https://console.groq.com/keys).
`DEFAULT_STT_PROVIDER`	No	`"groq"`	STT provider (`"groq"` or `"local"`).
`DEFAULT_GROQ_STT_MODEL`	No	`"whisper-large-v3-turbo"`	Whisper model identifier on Groq.
`DEFAULT_LLM_PROVIDER`	No	`"groq"`	LLM provider (`"groq"` or `"ollama"`).
`DEFAULT_GROQ_LLM_MODEL`	No	`"llama-3.3-70b-versatile"`	Groq LLM model for blueprint generation.
`OLLAMA_HOST`	No	`"http://localhost:11434"`	Local Ollama endpoint address.
`AUDIO_SAMPLE_RATE`	No	`16000`	Audio sampling frequency in Hz.

4. Local Installation & Launch Guide

```
# 1. Clone or navigate to the repository
cd voice_insights_app

# 2. (Optional) Create and activate virtual environment
python -m venv venv
venv\Scripts\activate  # On Windows
```

```
# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure environment
cp .env.example .env
# Edit .env and paste your GROQ_API_KEY

# 5. Launch the Graphical Desktop App
python main.py

# Or run in CLI mode
python main.py transcribe temp_audio/sample.wav
```

Chapter 6: UI/UX Design System & Aesthetics

UI/UX Design System & Aesthetics (design.md)

This document details the user experience principles, visual design tokens, component hierarchy, and audio visualization algorithms used in the **Voice-to-Insight** application.

---

1. Design Philosophy

- **Function-Driven Simplicity**: Every UI element directly supports one of three steps: **Capture *to* Ground *to* Generate**.
- **High-Contrast Dark Theme**: Tailored for developer environments, reducing eye fatigue during extended coding and architecture sessions.
- **Dynamic Feedback**: Real-time visual feedback during recording (waveform visualizer, recording clock, pulse indicators) ensures confidence that speech is being captured accurately.

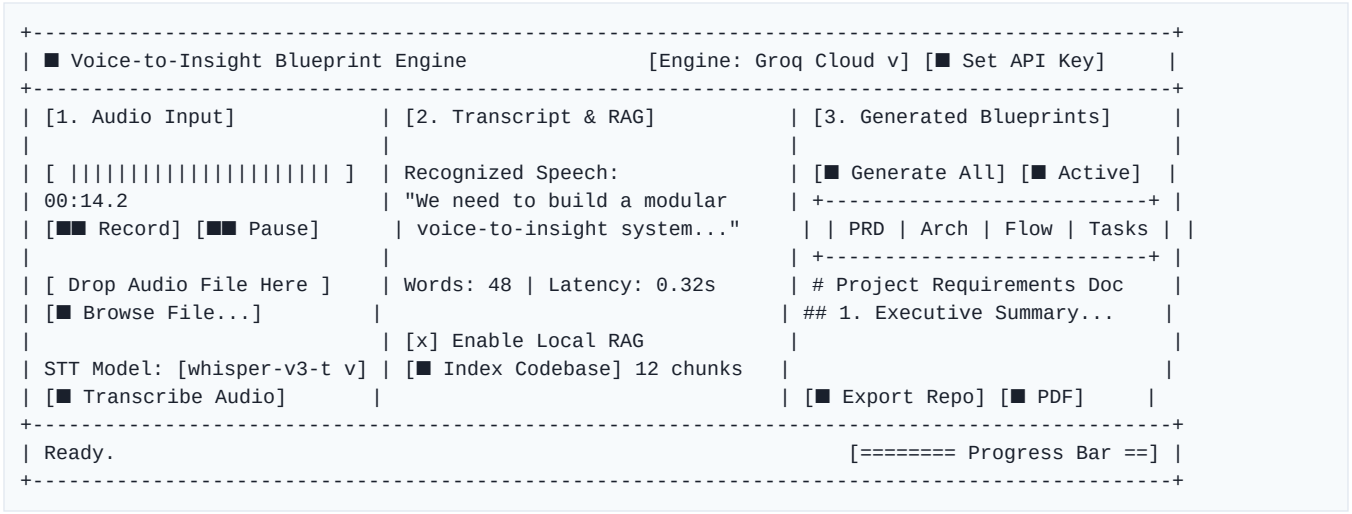
---

2. Color Palette & Design Tokens

Token Name	Hex Code	Purpose
`bg-primary`	`#14181E`	Deep slate window background
`bg-surface`	`#1A202C`	Card and panel container backgrounds
`bg-hover`	`#2D3748`	Interactive button and drop-zone hover states
`accent-blue`	`#3182CE`	Primary action buttons and focus rings
`accent-green`	`#276749`	Blueprint generation & success indicators
`accent-red`	`#9B2C2C`	Active recording indicator & stop action
`text-primary`	`#E2E8F0`	High-legibility headers and editor text
`text-muted`	`#718096`	Metadata labels, timestamps, and placeholders

---

3. Component Hierarchy & 3-Pane Layout



4. Audio Visualizer Mathematical Model

The multi-bar waveform visualizer maps real-time RMS audio power into a dynamic frequency spectrum display:

1. **RMS Power Calculation**:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x[i]^2}, \quad L_{\text{norm}} = \min(1.0, \text{RMS} \times 10.0)$$

2. **Spatial Bell-Curve Weighting**: Bars near the center receive higher weight to simulate human vocal resonance:

$$W(i) = \max\left(0.2, 1.0 - 0.7 \cdot \frac{|i - c|}{c}\right), \quad \text{where } c = \frac{N_{\text{bars}}}{2}$$

3. **Decay Physics**: When speech pauses, each bar falls exponentially at rate $\Delta L = -0.05$ every 30ms for natural fluid motion.

Chapter 7: Engineering Rules & Code Standards

Project Engineering Rules & Conventions (rules.md)

This document establishes development standards, code conventions, prompt engineering principles, and architectural guidelines for the **Voice-to-Insight** repository.

---

1. Python Code Standards

1. **Type Annotations**: All public functions and methods must include Python 3.10+ type hints (`typing.Optional`, `typing.List`, `typing.Dict`, `typing.Tuple`).

2. **Docstrings & Comments**: Every module, class, and public function must have clear docstrings explaining purpose, parameters, return values, and potential exceptions.

3. **Thread Safety**: Never perform network I/O, heavy file operations, or audio processing directly on the Qt main thread. Use `QThread` with custom `pyqtSignal` objects.

4. **Error Handling**: Use explicit `try-except` blocks. Never use bare `except:` without logging or bubbling up user-friendly error messages.

---

2. Audio & Signal Processing Rules

1. **Sample Rate Standardization**: All recorded or processed audio must be normalized to **16,000 Hz, 16-bit PCM mono**.

2. **Audio Buffer Cleanliness**: Always release audio streams (`stream.stop()`, `stream.close()`) in `finally:` blocks or when closing the application.

3. **Temporary File Hygiene**: Audio files stored in `temp_audio/` must use unique timestamped names to avoid write collisions.

---

3. RAG & Retrieval Rules

1. **Header-Aware Chunking**: Markdown files must be segmented by section headers (#, ##, ###) to preserve logical semantic context.

2. **Query Decomposition**: Free-form transcripts must be filtered for domain keywords and split into individual search queries before hitting the BM25 index.

3. **Grounding Prompts**: Injected context blocks must clearly state source filenames and chunk IDs for traceability.

4. Prompt Engineering & Markdown Output Rules

1. Valid Mermaid Diagrams: All generated architecture diagrams and sequence flows must follow strict, parseable Mermaid syntax (`flowchart TD`, `sequenceDiagram`, `stateDiagram-v2`).

2. Actionable Tasks: Task checklists must use standard GitHub Markdown checkbox formatting (`- [ ]`) with clear acceptance criteria.

3. No Conversational Fluff: System prompts must instruct the LLM to output pure Markdown without conversational intro or outro filler text (*"Sure, here is your PRD..."*).

Chapter 8: QA Test Checklists & Rollback Procedures

Quality Assurance, Test Checklists & Rollback Procedures

This document provides testing checklists, edge-case validation matrices, and disaster recovery / rollback plans for the **Voice-to-Insight** system.

---

1. Automated & Manual Test Checklist

A. Audio Recording Subsystem

[ ] Default Input Device Detection: Verify `sounddevice.query_devices()` identifies an active microphone on Windows/macOS/Linux.

[ ] Buffer Continuity: Verify no audio clipping or frame drops occur during a 60-second continuous recording session.

[ ] Pause & Resume: Verify that pausing the recording halts frame capture and resuming appends seamlessly without audio glitches or duration skew.

[ ] WAV File Integrity: Verify saved `.wav` files match standard specifications: 16,000 Hz, 16-bit PCM, single-channel mono.

[ ] Visualizer Responsiveness: Verify the `AudioVisualizer` widget renders dynamic amplitude bars during speech and decays gracefully to zero during silence.

B. Speech-to-Text (STT) Subsystem

[ ] Valid Groq API Key: Verify successful sub-second transcription of a sample 10-second `.wav` file.

[ ] Multi-Format Ingestion: Verify transcription works seamlessly across `.wav`, `.mp3`, `.m4a`, `.ogg`, `.flac`, and `.webm`.

[ ] Missing API Key Handling: Verify an informative user dialog is presented if `GROQ_API_KEY` is missing or empty.

[ ] Large Audio File Handling: Verify files up to 25 MB are handled cleanly without timeout or memory exhaustion.

C. RAG Knowledge Base Subsystem

[ ] Recursive Indexing: Verify `LocalRAGEngine.index_directory()` parses subdirectories and ignores binary/unsupported extensions.

[ ] Header-Aware Chunking: Verify `.md` files are split cleanly at #, ##, and ### headers.

[ ] BM25 Retrieval Scoring: Verify that searching for specific domain terms (e.g. `fastapi`, `database`, `jwt`) retrieves the exact matching code chunks as top 1.

[ ] Query Decomposition: Verify that a multi-sentence transcript is successfully decomposed into domain-specific subqueries.

D. Insight & Blueprint Generation Subsystem

[ ] PRD Generation: Verify `PRD.md` includes Vision, Goals, Functional Requirements (P0/P1/P2), and Non-Functional Requirements.

[ ] Architecture Generation: Verify `architecture.md` includes valid, parseable Mermaid diagrams (`flowchart TD` or `sequenceDiagram`).

[ ] Flow Generation: Verify `flow.md` details initialization order, call hierarchy, and state transitions.

[ ] Tasks Checklist: Verify `tasks.md` outputs standard Markdown checkboxes (`- [ ]`).

[ ] Provider Toggle: Verify seamless switching between Groq Cloud and Ollama Local without restarting the app.

---

2. Edge Case Matrix & Error Handling

Scenario / Edge Case	Expected System Behavior	Recovery / Mitigation
No Microphone Detected	`AudioRecorder.start()` catches `sd.PortAudioError`.	Displays critical QMessageBox: *"No microphone found. Please connect an audio input device."*
Silent Audio Recording (Mic Muted)	Whisper outputs empty string or warning token.	System detects empty transcript and prompts the user: *"No speech detected in audio."*
Network Timeout or 429 Rate Limit	`TranscriptionWorker` / `InsightWorker` catches HTTP exception.	Emits `error` signal to UI; displays user-friendly error dialog without crashing the application.
Target Directory Read-Only	`ExportService` catches `PermissionError`.	Alerts the user to select an alternate directory with write permissions.
Ollama Service Offline	`requests.exceptions.ConnectionError` on `http://localhost:11434`.	Prompts user: *"Could not connect to Ollama at localhost:11434. Please ensure Ollama is running (`ollama serve`)."*

---

3. Rollback & Disaster Recovery Procedures

Scenario A: API Key Invalidation or Cloud Outage

1. **Immediate Fallback**: Switch the **Engine** dropdown from *Groq Cloud* to *Ollama (Local LLM)*.
2. **Local Transcription**: Point the STT pipeline to local `faster-whisper` or local audio files.

3. **Data Preservation**: All recorded WAV files remain safely preserved inside the `temp_audio/` folder.

Scenario B: Corrupted Document Generation

1. Each tab in the UI is an editable `QTextEdit`. If an LLM response has formatting issues, users can click **"Generate Active Tab Only"** to re-synthesize that single document without losing progress on other tabs.
2. The user can manually edit the markdown directly in the tab before clicking **"Export All to Repository"**.

---

4. Diagnostic Commands

```
# Run unit test suite
python tests/test_stt.py
python tests/test_recorder.py
python tests/test_rag.py

# Test CLI transcription
python main.py transcribe temp_audio/sample.wav

# Test CLI end-to-end blueprint generation
python main.py generate --text "Build a high-performance voice-to-insight system with PyQt6 and Groq" --expor
t-dir ./output
```

Chapter 9: Market Comparison, Positioning & Commercialization

Market Comparison, Positioning & Commercialization Strategy (comparison.md)

This document provides a technical comparison between **Voice-to-Insight** and modern voice dictation tools (such as **Wispr Flow** and **Superwhisper**), detailing the paradigm shift from basic dictation to autonomous blueprint synthesis, proprietary intellectual property moats, and a commercialization roadmap.

---

1. The 3 Generations of Voice Interfaces

```
Gen 1: Raw Transcription (Basic Whisper / Apple Dictation / Otter.ai)
   ■ Voice ■■■ Verbatim Raw Text (Messy, filled with "umms", stutters, repetitions)

Gen 2: Smart Dictation (Wispr Flow / Superwhisper)
   ■ Voice ■■■ Cleaned Text & Formatted Paragraphs (Pasted into active cursor)
   ■ Focus: Saves typing keystrokes for general writing, messaging, and emails.

Gen 3: Cognitive Action & Architectural Synthesis (Voice-to-Insight)
   ■ Voice ■■■ Decompose Intent ■■■ RAG Codebase Grounding ■■■ 6-Doc Engineering Blueprint Suite + Mermaid
 Diagrams + PDF
   ■ Focus: Saves hours of engineering planning, PRD authoring, and system architecture design.
```

---

2. Feature-by-Feature Comparison: Voice-to-Insight vs. Wispr Flow

Dimension	**Wispr Flow (Gen 2: Smart Dictation)**	**Voice-to-Insight (Gen 3: Cognitive Synthesis)**
Primary Utility	Faster typing in chat boxes, emails, and notes.	Autonomous software architecture, PRD authoring, and task automation.
Output Format	Single text block / auto-formatted paragraph inserted at cursor.	**6 Standardized Engineering Artifacts** (`PRD.md`, `architecture.md`, `flow.md`, `tasks.md`, `tech_stack.md`, `implementation_plan.md`) + **Master PDF**.
Codebase Awareness	**None.** Operates in a vacuum without knowledge of real schemas or code.	**Deep Codebase Grounding (RAG).** Scans `.py`, `.ts`, `.md`, `.json` repositories using Okapi BM25 to prevent hallucinations.

Visual Architecture	None (text-only).	**Generates parseable Mermaid diagrams** (System flowcharts, sequence diagrams, state machines).
Latency & Hardware	Cloud WebSockets to Whisper (~1–2s).	**Groq LPU Acceleration**: Sub-second transcription (200–400ms) with zero client GPU overhead.
Execution Flexibility	Cloud-only proprietary SaaS.	**Dual Engine**: High-speed Groq Cloud + **100% Offline/Private Ollama Local Fallback**.
Target User Base	General office workers, writers, and casual communicators.	Software engineers, system architects, technical founders, and dev agencies.

---

3. How the Pipeline Operates Under the Hood

```
flowchart TD
    subgraph Wispr_Flow_Pipeline ["Wispr Flow Dictation Pipeline"]
        A1["Spoken Speech"] --> B1["Cloud Whisper STT"]
        B1 --> C1["LLM Formatting (Filler Word Removal)"]
        C1 --> D1["Clean Text inserted into active window"]
    end

    subgraph Voice_to_Insight_Pipeline ["Voice-to-Insight Architectural Pipeline"]
        A2["Spoken Brain-Dump"] --> B2["Groq Whisper LPU (Sub-second)"]
        B2 --> C2["Spoken Query Decomposition"]
        C2 --> D2["Local Codebase RAG (Okapi BM25 Index)"]
        D2 --> E2["Multi-Agent Prompt Synthesizer"]
        E2 --> F2["PRD, Architecture (Mermaid), Flows, Tasks, Tech Stack & PDF"]
        F2 --> G2["1-Click Git Repository Export"]
    end
```

The Cognitive Difference: A Real-World Example

Suppose an engineer speaks for 30 seconds:

> *"Hey, let's add an audio caching layer to our STT pipeline so if the same user uploads a file twice we don't hit the Groq API again. Store the hash in Redis or SQLite and add a task to write unit tests for it."*

- **In Wispr Flow**:

  Cleans the grammar and types:

  "Let's add an audio caching layer to our STT pipeline to avoid re-querying the Groq API for duplicate files, using Redis or SQLite for hash storage, along with unit tests."

  (The user still has to manually design the architecture, write the PRD, create the task list, and draw sequence diagrams).

- **In Voice-to-Insight**:

  1. Searches the local repository to see how files, buffers, and hashes are handled.

  2. Generates `PRD.md` *with problem statements, functional requirements, and cache invalidation policies.*

  3. Generates `architecture.md` *with a Mermaid diagram:*

`Audio Ingestion ■■■ SHA-256 Hash ■■■ Cache Lookup ■■■ [Hit: Instant Return] / [Miss: Groq LPU]`.

4. Generates `tasks.md` *with a prioritized checklist:*

-[ ] `Phase 1: Implement SHA-256 audio buffer hashing in audio_recorder.py`

-[ ] `Phase 2: Add SQLite/Redis cache lookups in stt_service.py`

-[ ] `Phase 3: Write cache hit/miss unit tests in tests/test_stt.py`

5. Exports all files directly to the repository and compiles the ***Master PDF****.*

---

4. Commercialization & Business Models

```
flowchart TD
    CoreEngine["Voice-to-Insight Core Engine"]

    CoreEngine --> ModelA["Model A: Prosumer Desktop SaaS<br/>($19 - $29/mo)"]
    CoreEngine --> ModelB["Model B: B2B Engineering Teams<br/>($49 - $99/seat/mo)"]
    CoreEngine --> ModelC["Model C: Agency & Freelancer Pro<br/>($99 - $199/mo)"]

    ModelA --> TargetA["Indie Hackers, Solo Devs, Tech Leads"]
    ModelB --> TargetB["Startups, Jira/Linear Teams, Remote Companies"]
    ModelC --> TargetC["Software Dev Agencies, Consultants"]
```

Model A: Prosumer Desktop SaaS ("Cursor for Voice Architecture")

- **Target Audience**: Solo developers, architects, indie hackers, and technical founders.
- **Pricing Tier**: ***19 – 29 / month***.
- **Key Features**:
- Global hotkey shortcut on Windows/macOS.
- Automatic codebase RAG indexing.
- 1-Click export to local Git repos and Markdown files.

Model B: B2B Engineering & Sprint Planning Suite

- **Target Audience**: Startups and enterprise engineering teams using Jira, Linear, GitHub, and Notion.
- **Pricing Tier**: ***49 – 99 / seat / month***.
- **Key Features**:
- Ingests Zoom/Google Meet architecture discussions.
- Automatically extracts architectural decisions and **creates Jira/Linear epics and GitHub issues**.
- Team-wide synchronized codebase schemas and architecture standards.

Model C: Software Agency & Client Discovery Accelerator

- **Target Audience**: Digital product agencies, dev shops, and freelance technical consultants.

- **Pricing Tier**: ***99 – 199 / month***.
- **Key Features**:
- Ingests 30-minute client discovery calls.
- Generates instant branded PDF project proposals, technical architecture specifications, and task/cost breakdowns in 30 seconds.

---

5. Proprietary Intellectual Property Moats

To establish defensibility against generic AI wrappers:

1. **Proprietary Multi-Agent Planning Pipelines**:
   - Algorithmic translation of non-linear spoken disfluencies into validated, syntax-checked Mermaid diagrams, schema contracts, and Agile tasks.
2. **Deep Code Graph RAG**:
   - Hybrid indexing combining Abstract Syntax Tree (AST) code structures, dependency graphs, and Okapi BM25 semantic retrieval.
3. **IDE-Native Plugins (VS Code, JetBrains, Cursor)**:
   - Voice-activated architecture assistant embedded directly inside the developer's active workspace.
4. **Enterprise Air-Gapped Privacy Mode**:
   - Zero-data-retention on-premise execution using local Ollama models for high-security fintech, healthcare, and defense clients.

---

6. Financial Unit Economics & Profit Margins

Using Groq LPU infrastructure yields extraordinary profit margins:

Expense Layer	Estimated Cost	User Subscription	Gross Margin
STT (Groq Whisper)	~$0.04 / hour of audio	—	—
LLM (Groq Llama-3.3-70B)	~$0.005 / full blueprint suite	—	—
Total Monthly Cost Per Active User	***~0.40* – 0.80 / month**	**$20.00 / month**	**> 96% Gross Margin**

The negligible API cost structure allows offering a generous free tier for viral adoption while capturing high-margin subscription revenue on paid tiers.